

DISASTER RESOURCE RESPONSE OPTIMIZER

Complete Final-Year Project Document

Technology Stack: Java / Spring Boot / React / MySQL

Planned implementation target: 30 September 2026

1. Executive Summary

Disaster Resource Response Optimizer (DRRO) is a web-based decision-support platform designed to help disaster-response organizations allocate limited emergency resources to affected locations more efficiently. The system combines disaster information, relief requests, resource availability, responder availability, location/distance information and an optimization-based priority engine to recommend where available resources should be sent.

The project is deliberately positioned as more than a conventional disaster-management CRUD application. Its central contribution is an explainable allocation engine that ranks requests and recommends allocations using multiple factors such as severity, population affected, urgency, resource shortage, estimated travel distance/time and available supply. The system also records the reasoning behind recommendations so that a response officer can review, approve, modify or reject them.

The academic motivation is supported by emergency-management research showing that disaster response is a spatio-temporal decision problem under uncertainty, and by recent work on robust, multi-period, equitable and integrated resource allocation. The proposed student project is a practical, smaller-scale implementation inspired by these research directions rather than a claim to replace professional emergency-management systems.

2. Project Title

Disaster Resource Response Optimizer (DRRO): An Explainable Decision-Support System for Dynamic Allocation of Limited Disaster-Response Resources

3. Background

During floods, earthquakes, cyclones, landslides, wildfires and similar emergencies, multiple affected locations can request food, water, medicines, rescue equipment, vehicles, shelters and personnel at the same time. The available supply is often limited, transportation capacity is constrained and the situation can change rapidly.

Operational disaster logistics therefore requires more than maintaining an inventory. Emergency-management organizations need to identify requirements, identify resources, balance supply against requirements, coordinate transport and track the response. FEMA's logistics framework, for example, explicitly describes identifying logistics requirements and resources and balancing resources with requirements as core resource-management activities.

Research literature similarly treats emergency response as decision-making under uncertainty involving incident prediction, detection, resource allocation and computer-aided dispatch. Recent research continues

to investigate multi-stage robust optimization, uncertainty, equity, routing and integrated personnel/material allocation.

4. Problem Statement

Existing basic disaster-management applications generally record disasters, victims, resources and requests but do not provide an intelligent mechanism for deciding which request should receive scarce resources first. A manual or first-come-first-served approach may allocate supplies without adequately considering severity, population affected, urgency, shortage, distance, travel time and resource availability.

The proposed system addresses this problem by creating a centralized platform that continuously compares demand and supply and generates an explainable allocation recommendation. The recommendation is not an autonomous command: an authorized officer remains responsible for approval and operational decisions.

5. Aim

To design and implement a web-based disaster-response decision-support system that improves the prioritization and allocation of limited emergency resources by combining multi-factor scoring, resource constraints, location-aware information and allocation optimization.

6. Objectives

- Maintain centralized information about disasters, affected locations, resource inventories, requests and response teams.
- Allow authorized personnel to create, update and monitor disaster incidents and relief requests.
- Track resource quantity, location, availability and allocation status.
- Calculate a transparent priority score for each relief request.
- Recommend suitable resource sources based on need, supply, distance and constraints.
- Reduce response time and avoid inefficient distribution of scarce resources.
- Track partial fulfillment and unmet demand rather than treating a request as simply fulfilled/unfulfilled.
- Provide dashboards and reports for operational monitoring and project evaluation.
- Measure the proposed method against simpler allocation baselines such as first-come-first-served and nearest-source allocation.
- Provide an academically defensible research contribution through explainable multi-factor allocation and experimental evaluation.

7. Scope

7.1 In Scope

- Natural-disaster scenarios such as floods, earthquakes, cyclones and landslides.
- Multiple disasters and multiple affected locations.
- Multiple resource categories and resource centers.
- Relief requests with quantity and urgency.
- Resource inventory and allocation tracking.
- Priority scoring and allocation recommendation.

- Distance/travel-time factor using stored coordinates or an optional map/routing service.
- Responder/team assignment.
- Officer approval and allocation status tracking.
- Dashboard, reports, audit history and performance metrics.
- Simulation/testing using historical or synthetic disaster scenarios.

7.2 Out of Scope for the First Version

- Autonomous dispatch of real emergency vehicles.
- Guaranteed real-time satellite/IoT integration.
- Replacement of government emergency command systems.
- Guaranteed prediction of where a disaster will occur.
- Medical diagnosis or clinical decision-making.
- Fully autonomous routing in dangerous or changing road conditions.
- Production-grade nationwide deployment.

8. Target Users and Roles

- System Administrator – manages users, roles, master data and system configuration.
- Disaster Management Officer – registers incidents, verifies requests and reviews recommendations.
- Resource Manager – manages inventories, warehouses/resource centers and allocation status.
- Response Team Coordinator – manages teams, skills, availability and assignments.
- Field/Relief Operator – updates delivery, arrival and fulfillment information.
- Viewer/Analyst – accesses dashboards and reports according to permissions.

9. Proposed System

DRRO follows a layered web architecture. The React frontend communicates with a Java Spring Boot REST API. Spring Boot handles authentication, business rules, request validation, allocation services and reporting. MySQL stores operational data. The allocation engine is implemented as a separate service/module so that the algorithm can be tested independently.

The system uses human-in-the-loop decision support: the algorithm recommends an allocation, displays the factors used, and allows an authorized officer to approve or modify the recommendation. This design improves explainability and reduces the risk of treating an experimental algorithm as an unquestionable operational authority.

10. Major Functional Modules

10.1 Authentication and Role Management

- Login/logout and secure password handling.
- Role-based access control.
- User activation/deactivation.
- Audit trail for important actions.

10.2 Disaster Management

- Create disaster incident.

- Record disaster type, date/time, location, severity and description.
- Maintain incident status: ACTIVE, CONTAINED, RECOVERING, CLOSED.
- Link affected zones and requests to a disaster.

10.3 Affected Location Management

- Store location name and coordinates.
- Population affected and vulnerable-population indicators.
- Severity and accessibility information.
- Number of open requests.
- Current fulfillment status.

10.4 Resource Management

- Create resource types and units.
- Register warehouses/resource centers.
- Track available, reserved, dispatched and delivered quantities.
- Track expiry for applicable resources such as medicines and food.
- Maintain minimum stock/reorder thresholds.

10.5 Relief Request Management

- Create request for one or more resource types.
- Record required quantity, urgency, deadline and affected population.
- Validate duplicate/open requests.
- Track partial fulfillment.
- Escalate unmet requests.

10.6 Resource Allocation Engine

- Collect open requests and available supply.
- Calculate request priority.
- Identify feasible resource sources.
- Estimate distance/travel time.
- Generate an allocation recommendation.
- Support partial allocation when supply is insufficient.
- Record the factors behind each recommendation.

10.7 Response Team Management

- Maintain team profile, location, skills and availability.
- Assign teams to incidents/locations.
- Track assignment and completion status.

10.8 Dispatch and Delivery Tracking

- Create dispatch order.
- Assign resource source and team/vehicle.
- Record departure, estimated arrival and actual arrival.
- Update delivered quantity.
- Automatically update inventory and request fulfillment.

10.9 Dashboard and Reporting

- Active disasters.
- High-priority requests.
- Available versus required resources.
- Unmet demand.
- Allocation history.
- Average response time.
- Resource utilization.
- Allocation comparison metrics.

11. Core Innovation / Research Contribution

The project should not be presented as the invention of disaster-resource optimization itself. Emergency resource allocation is an established research field. The contribution of the student project is the design and evaluation of a practical, explainable, human-in-the-loop allocation framework that can be implemented as a web application and compared experimentally with simple baseline strategies.

The proposed contribution has four parts:

1. Explainable Multi-Factor Priority: combine severity, affected population, urgency, shortage and distance/travel-time into a transparent score.
2. Constraint-Aware Allocation: prevent recommendations from exceeding available inventory and account for source/request compatibility and optional transport limits.
3. Dynamic Recalculation: when supply, demand or request status changes, recalculate priorities and recommendations instead of relying on a fixed allocation.
4. Experimental Evaluation: compare DRRO against first-come-first-served, severity-only and nearest-source baselines using response time, unmet demand, distance/cost, utilization and fairness indicators.

This direction is consistent with the research landscape. Recent studies have explored multi-stage robust resource allocation, uncertainty in demand and transportation, multi-period allocation, equity, and integrated personnel/material decisions. These studies support the importance of the problem but do not make the proposed student application itself novel by default.

12. Proposed Priority Model

For a first implementable version, normalize each factor to a 0–100 scale and calculate:

$$\text{Priority Score} = wS*S + wP*P + wU*U + wD*D + wT*T + wV*V$$

- S = disaster/location severity score.
- P = affected population/vulnerability score.
- U = urgency/deadline score.
- D = resource deficit/shortage score.
- T = travel-time or accessibility urgency score.
- V = vulnerability/critical-needs score.
- wS...wV = configurable weights whose sum equals 1.

Example starting weights for experimentation: Severity 0.25, Population/Vulnerability 0.20, Urgency 0.20, Resource Shortage 0.20, Travel/Accessibility 0.10 and Critical-Need factor 0.05. These are proposed starting values, not scientifically established constants. The project should test alternative weights through sensitivity analysis.

A separate feasibility layer should ensure that the algorithm does not recommend more stock than is available, does not allocate incompatible resource types, and does not assign inactive sources.

13. Allocation Logic

5. Fetch all active and verified requests.
6. Fetch available resource quantities from eligible resource centers.
7. Normalize request factors.
8. Calculate priority scores.
9. Sort requests by priority or solve the allocation optimization model.
10. For each request, identify feasible resource sources.
11. Estimate source-to-destination distance/travel time.
12. Allocate full quantity if feasible; otherwise allocate the maximum reasonable partial quantity.
13. Update reserved/allocated quantities.
14. Create an explainable recommendation record.
15. Send recommendation to an authorized officer.
16. After approval, create dispatch and update inventory.
17. After delivery, update delivered quantity and request status.
18. Recalculate remaining unmet demand.

14. Optimization Formulation

A simplified formulation can use x_{ij} as the quantity of resource sent from source i to demand point j . Let c_{ij} represent transportation cost/time, p_j represent request priority, and d_j represent remaining demand.

Illustrative objective:

Maximize: $\sum(p_j * x_{ij}) - \alpha \sum(c_{ij} * x_{ij}) - \beta \sum(\text{unmet}_j)$

Subject to:

- $\sum_j x_{ij} \leq \text{supply}_i$ for every source i .
- $\sum_i x_{ij} \leq \text{demand}_j$ for every request j .
- $x_{ij} \geq 0$.
- $x_{ij} = 0$ when source i cannot provide the requested resource.
- Optional vehicle capacity and route constraints.
- Optional minimum-service constraints for critical requests.

For the first implementation, the allocation can be solved using a greedy priority-aware algorithm. If time permits, a linear-programming solver can be added as an advanced comparison. The project should not depend on a complex solver to remain achievable by 30 September 2026.

15. Baseline Algorithms for Research Evaluation

- Baseline A – First-Come-First-Served: fulfill requests according to creation time.

- Baseline B – Severity Only: sort by disaster/location severity.
- Baseline C – Nearest Source: choose the closest feasible resource center.
- Proposed – DRRO Multi-Factor Allocation: use priority, shortage, distance and supply constraints.

This comparison is important because the project needs measurable evidence that the proposed approach is useful rather than merely more complicated.

16. Database Design

Recommended core tables:

Table	Purpose	Important Fields	Relationships
users	System users	user_id, name, email, password_hash, role, status	roles/audit
roles	Access roles	role_id, role_name	users
disasters	Incident records	disaster_id, type, severity, start_time, status	locations/requests
locations	Affected/resource locations	location_id, name, latitude, longitude, population, vulnerability	disasters/requests/resources
resource_types	Resource master data	resource_type_id, name, unit, category	inventory/requests
resource_centers	Warehouses/resource centers	center_id, location_id, name, status	inventory
inventory	Available stock	inventory_id, center_id, resource_type_id, available_qty, reserved_qty	centers/resource types
relief_requests	Demand records	request_id, disaster_id, location_id, urgency, status, created_at	request_items
request_items	Resource demand lines	request_item_id, request_id, resource_type_id, required_qty, fulfilled_qty	requests/resource types
allocations	Approved allocation	allocation_id, request_item_id, center_id, quantity, score, status	requests/inventory
allocation_factors	Explainability	allocation_id, severity_score, urgency_score, shortage_score, distance_score	allocations
response_teams	Teams	team_id, name, skills, location_id, availability	assignments
dispatches	Transport/delivery	dispatch_id, allocation_id, team_id, vehicle, dispatched_at, delivered_at	allocations/teams
audit_logs	Traceability	log_id, user_id, action,	users

		entity, entity_id, timestamp	
--	--	---------------------------------	--

17. Entity Relationship Structure

Core relationships:

- One disaster can have many affected locations and many relief requests.
- One relief request can contain many request items.
- One resource type can appear in many inventory records and request items.
- One resource center can hold many inventory records.
- One request item can receive multiple allocations, enabling partial fulfillment.
- One allocation can create one or more dispatch records depending on the chosen operational design.
- One response team can receive many assignments over time but should normally have only compatible active assignments.
- Users generate audit records for important actions.

18. System Architecture

Presentation Layer: React.js pages, forms, dashboards, tables and charts.

API Layer: Spring Boot REST controllers for authentication, disasters, locations, resources, requests, allocation, teams, dispatch and reports.

Business Layer: service classes containing validation, workflow rules, priority calculation and allocation logic.

Data Layer: Spring Data JPA/Hibernate repositories connected to MySQL.

Optimization Layer: independent AllocationEngine service with interfaces that allow a greedy algorithm first and an LP/optimization implementation later.

External Integration Layer (optional): map/routing API, email/SMS notification provider, weather or open-data source. External integrations should be optional so the core project remains demonstrable without them.

19. Suggested Backend Package Structure

com.drro

- config – security and application configuration
- controller – REST controllers
- dto – request/response DTOs
- entity – JPA entities
- repository – Spring Data repositories
- service – business services
- allocation – scoring and optimization engine
- security – authentication/authorization
- exception – global exception handling
- mapper – DTO/entity mapping

- util – reusable utilities
- report – report generation and metrics

20. Suggested React Frontend Structure

- pages/Login
- pages/Dashboard
- pages/Disasters
- pages/Locations
- pages/Resources
- pages/Requests
- pages/AllocationRecommendations
- pages/Teams
- pages/Dispatch
- pages/Reports
- components/Navbar, Sidebar, Tables, Forms, StatusBadge, PriorityCard
- services/api.js
- hooks/
- context/ or state management/
- utils/

21. Important Screens

19. Login page.
20. Admin dashboard.
21. Disaster registration page.
22. Disaster details page.
23. Affected-location page.
24. Resource-center inventory page.
25. Create relief request page.
26. Pending requests dashboard.
27. Allocation recommendation page showing score breakdown.
28. Approval/modification page.
29. Dispatch tracking page.
30. Response team page.
31. Reports and analytics page.

22. Example Allocation Explanation

Suppose three locations request drinking water. Location A has very high severity, a large affected population and severe shortage but is moderately far away. Location B is closer but has lower urgency. Location C is nearby but has enough remaining stock. The system should not simply choose the nearest location. It should score the needs and recommend an allocation that balances urgency and logistics.

The UI should show an explanation such as: 'High priority because severity is high, remaining supply covers only 25% of required quantity, and the affected population is large. Recommended source: Center 2 because it has sufficient stock and the estimated travel time is lower than other feasible sources.'

23. Functional Requirements

- FR-01: User can authenticate securely.
- FR-02: Admin can manage roles and users.
- FR-03: Officer can create and update disaster incidents.
- FR-04: Officer can register affected locations.
- FR-05: Resource manager can manage resource centers and inventory.
- FR-06: Officer can create and verify relief requests.
- FR-07: System calculates request priority.
- FR-08: System generates feasible allocation recommendations.
- FR-09: Officer can approve, reject or modify recommendations.
- FR-10: System tracks dispatch and delivery.
- FR-11: System updates inventory after allocation/delivery.
- FR-12: System records partial fulfillment and unmet demand.
- FR-13: System provides dashboards and reports.
- FR-14: System maintains audit logs.

24. Non-Functional Requirements

- Security: passwords hashed; role-based access; protected APIs.
- Performance: normal dashboard/API operations should respond quickly on the project test dataset.
- Scalability: database and allocation service should support multiple requests and resource centers.
- Reliability: transactions should prevent inconsistent inventory updates.
- Maintainability: layered architecture and modular allocation engine.
- Usability: clear forms, status indicators and explainable recommendations.
- Auditability: important allocation and approval actions must be traceable.
- Availability: deployable on a normal student laptop/local server and optionally cloud-hosted.

25. Security Design

- Use Spring Security.
- Store password hashes, never plain-text passwords.
- Use role-based endpoint authorization.
- Validate all input server-side.
- Use DTOs rather than exposing sensitive entities directly.
- Use transactions for allocation and inventory updates.
- Record audit logs for approval, rejection, allocation and stock changes.
- Do not expose unnecessary personal data.

26. Testing Strategy

26.1 Unit Testing

- Priority-score calculation.
- Shortage calculation.
- Allocation feasibility.
- Inventory deduction.

- Partial fulfillment logic.
- Status transitions.

26.2 Integration Testing

- Controller-service-repository flows.
- Authentication and role permissions.
- Allocation transaction with MySQL.
- Dispatch and delivery workflow.

26.3 System Testing

- Complete disaster-to-delivery workflow.
- Multiple simultaneous requests.
- Insufficient stock.
- Multiple resource centers.
- Partial delivery.
- Closed/invalid resource source.

26.4 Algorithm Evaluation

Run the same scenario with each baseline and the proposed method. Record total travel distance/time, unmet demand, weighted priority satisfaction, resource utilization, average response time and a fairness indicator.

27. Evaluation Metrics

- Average Response Time = average time from verified request to dispatch/delivery.
- Unmet Demand = total required quantity minus fulfilled quantity.
- Fulfillment Rate = $\text{fulfilled quantity} / \text{required quantity} \times 100$.
- Resource Utilization = $\text{allocated quantity} / \text{usable available quantity} \times 100$.
- Transportation Cost/Distance = total distance or estimated cost of allocation.
- Priority Satisfaction = proportion/weighted amount of high-priority demand served.
- Equity/Fairness = compare fulfillment across affected locations; a Gini-style measure can be explored if time permits.
- Computation Time = time taken by each algorithm to generate recommendations.

28. Research Questions

- RQ1: Does multi-factor prioritization reduce unmet high-priority demand compared with first-come-first-served?
- RQ2: Does incorporating distance/travel time reduce transportation effort compared with severity-only allocation?
- RQ3: Does dynamic recalculation improve fulfillment when supply or demand changes?
- RQ4: What happens to efficiency and fairness when the priority weights are changed?
- RQ5: Can an explainable greedy approach provide useful decisions within acceptable computation time for a student-scale disaster scenario?

29. Hypotheses

- H1: DRRO will achieve a higher weighted fulfillment of high-priority requests than first-come-first-served.
- H2: DRRO will reduce unnecessary transportation distance compared with purely need-only allocation.
- H3: Dynamic recalculation will reduce unmet demand after changes in inventory or requests.
- H4: Weight selection creates a measurable trade-off between efficiency and equity.

30. Existing Research and Related Work

Emergency resource allocation is a well-established research field. A major review describes emergency response management as spatio-temporal decision-making under uncertainty and surveys incident prediction, detection, resource allocation and computer-aided dispatch. Recent research continues to extend the field into robust, multi-stage and multi-objective models.

A 2021 study proposed multiperiod cross-regional emergency resource allocation targeting delivery time, allocation cost and coverage/equity. Later research examined equitable and efficient allocation under uncertainty, including uncertainty in demand, transportation time and transport capacity. Recent 2026 work has investigated multi-stage robust allocation and integrated resource flows, while other work has considered uncertainty in demand, transportation costs and disruption risk.

These studies demonstrate that the underlying optimization problem is legitimate and research-active. The proposed project deliberately adopts a simpler, implementable architecture and uses these research directions to justify its evaluation criteria rather than claiming to reproduce state-of-the-art mathematical optimization.

31. Existing Systems / Real-World Inspiration

Professional emergency-management organizations already perform logistics and resource management. FEMA's logistics material describes balancing requirements and resources and managing the movement of supplies. FEMA's more recent response-and-recovery material also describes using disaster requirements, location, quantity and cost when making sourcing recommendations. DRRO should therefore be positioned as an academic prototype inspired by real operational needs, not as a replacement for these professional systems.

The project can also be informed by public disaster datasets, simulated incidents and published case studies. For a student implementation, synthetic scenarios are recommended because they allow controlled comparison between algorithms without requiring access to confidential operational data.

32. Why the Project Is Different from a Normal CRUD Project

- Contains an algorithmic decision engine.
- Supports constrained resource allocation rather than only data storage.
- Produces measurable optimization outcomes.
- Stores the explanation behind recommendations.
- Supports dynamic re-prioritization.
- Can compare multiple allocation strategies experimentally.
- Creates a clear bridge between software engineering and operations-research concepts.

33. Advantages

- Improves visibility of supply and demand.
- Helps officers prioritize scarce resources.
- Supports partial fulfillment.
- Provides explainable recommendations.
- Reduces dependence on simple first-come-first-served allocation.
- Provides measurable research metrics.
- Modular architecture allows future optimization methods.
- Suitable for a Java full-stack final-year project.

34. Limitations

- Real disaster data can be incomplete or delayed.
- Travel time may change because roads can be blocked.
- Priority weights are subjective unless calibrated using real operational data.
- Optimization quality depends on the accuracy of input information.
- A student prototype cannot model every operational constraint.
- External map, weather and notification services may require APIs and internet access.

35. Future Enhancements

- Machine-learning demand forecasting.
- Real-time weather and hazard feeds.
- Road-blockage and dynamic routing integration.
- IoT-based warehouse/resource monitoring.
- Mobile application for field teams.
- SMS/WhatsApp/email alerts through approved services.
- Advanced linear/mixed-integer optimization.
- Robust or stochastic optimization for uncertain demand.
- Multi-disaster simultaneous optimization.
- Inter-agency resource sharing.
- Digital maps and GIS visualization.
- Blockchain-style tamper-evident logs only if a genuine requirement is identified.

36. Deployment Plan

- Development: local laptop using Java, Spring Boot, React and MySQL.
- Version control: Git/GitHub.
- Backend deployment: Spring Boot JAR/container.
- Frontend deployment: production React build.
- Database: MySQL.
- Optional cloud deployment: one backend service, one frontend service and managed MySQL.
- Use environment variables for database credentials and API keys.

37. Suggested Git Repository Structure

disaster-resource-response-optimizer/

- backend/
- frontend/
- database/
- docs/
- test-data/
- research/
- README.md

docs/ can contain architecture diagrams, API documentation, database diagrams, screenshots and the final report. research/ can contain experiment datasets, baseline results and plots.

38. Implementation Roadmap to 30 September 2026

The schedule below is designed for a student team and assumes regular weekly work.

Period	Main Work	Deliverable	Priority
15–21 Aug	Requirements, use cases, architecture, database design	SRS + ER design + Git repository	Critical
22–28 Aug	Spring Boot setup, MySQL, authentication, users/roles	Working backend foundation	Critical
29 Aug–4 Sep	Disaster, location, resource-center and inventory modules	Core CRUD APIs + UI	Critical
5–11 Sep	Relief requests, request items, validation	Request workflow	Critical
12–18 Sep	Priority engine + allocation algorithm + explanation	Working optimizer	Critical
19–23 Sep	Approval, dispatch, teams, delivery tracking	End-to-end workflow	Critical
24–26 Sep	Dashboard, reports, charts, algorithm experiments	Evaluation module	High
27–28 Sep	Testing, bug fixing, security, performance	Stable release candidate	Critical
29 Sep	Final dataset, screenshots, diagrams, paper/report	Documentation complete	Critical
30 Sep	Final integration, demo rehearsal and submission	Final project build	Critical

39. Minimum Viable Product (MVP)

- Login with roles.
- Create disaster.
- Create affected location.

- Create resource center and inventory.
- Create relief request.
- Calculate priority score.
- Generate allocation recommendation.
- Approve allocation.
- Update inventory.
- Record delivery.
- Show dashboard.

If time becomes limited, complete the MVP first. Advanced maps, ML, notifications and sophisticated optimization should only be added after the end-to-end workflow works reliably.

40. Recommended Development Order

32. Create Git repository and project documentation.
33. Design MySQL schema.
34. Create Spring Boot project.
35. Implement entities and repositories.
36. Implement authentication and roles.
37. Implement disaster/location APIs.
38. Implement resource/inventory APIs.
39. Implement relief-request APIs.
40. Implement allocation engine.
41. Create allocation recommendation UI.
42. Implement approval and inventory transactions.
43. Implement team/dispatch/delivery.
44. Build dashboard and reports.
45. Add experiments and baseline algorithms.
46. Test and document.

41. Sample API Design

Method	Endpoint	Purpose
POST	/api/auth/login	Authenticate user
GET	/api/disasters	List disasters
POST	/api/disasters	Create disaster
GET	/api/resources/inventory	View inventory
POST	/api/resources/inventory	Add/update inventory
GET	/api/requests/open	View open requests
POST	/api/requests	Create relief request
POST	/api/allocation/recommend	Generate recommendations
GET	/api/allocation/recommendations	View recommendations
POST	/api/allocation/{id}/approve	Approve allocation
POST	/api/dispatch	Create dispatch
POST	/api/dispatch/{id}/deliver	Record delivery
GET	/api/reports/metrics	Get evaluation/operational metrics

42. Example Use Case

Use Case: Allocate Water to a Flood-Affected Area

47. Officer registers a flood incident.
48. Officer registers three affected locations.
49. Each location submits water requirements.
50. Resource manager records water inventory at two centers.
51. System verifies open requests.
52. Allocation engine calculates scores.
53. Engine identifies feasible centers and estimates distance.
54. System recommends allocation.
55. Officer reviews factor breakdown and approves.
56. Inventory is reserved and dispatch is created.
57. Team delivers water.
58. Delivered quantity is recorded.
59. Remaining demand stays open and is reconsidered during the next allocation cycle.

43. Sample Test Scenario

Scenario: 3 affected locations, 2 resource centers, 1 resource type, total demand 10,000 units and available stock 7,000 units.

- Location A: demand 4,000; severity 90; population 5,000; urgency 90.
- Location B: demand 3,000; severity 70; population 3,000; urgency 70.
- Location C: demand 3,000; severity 50; population 2,000; urgency 60.
- Total demand exceeds supply, so the algorithm must prioritize rather than fully satisfy all requests.
- Run all four strategies and compare fulfillment, unmet demand and transportation distance.

44. Expected Results

- High-priority requests receive a greater share of scarce resources.
- Inventory constraints are respected.
- Allocation decisions become traceable and explainable.
- Officers can see unmet demand rather than losing it in manual records.
- Compared with simple baselines, the proposed approach should demonstrate improvements on at least some selected metrics; the actual result must be established experimentally and not assumed beforehand.

45. Project Deliverables

- Software requirements specification.
- Architecture diagram.
- ER diagram and database script.
- Spring Boot backend.
- React frontend.
- MySQL database.
- Allocation algorithm.

- Baseline algorithms.
- Test cases and test results.
- Experiment dataset.
- Performance comparison.
- Final project report.
- Research paper draft.
- Presentation/PPT.
- Demo video or screenshots if required.

46. Suggested Final Report Chapters

60. Introduction
61. Problem Statement and Objectives
62. Literature Survey
63. Existing System
64. Proposed System
65. Requirements Analysis
66. System Architecture
67. Database Design
68. Module Design
69. Allocation Algorithm
70. Implementation
71. Testing
72. Experimental Evaluation
73. Results and Discussion
74. Conclusion
75. Future Work
76. References

47. Suggested Literature Survey Themes

- Emergency resource allocation.
- Humanitarian logistics.
- Disaster response optimization.
- Vehicle routing in disaster relief.
- Multi-period allocation.
- Robust/stochastic optimization.
- Equity and fairness in relief allocation.
- Explainable decision support.
- Dynamic resource allocation.

48. Important Academic Positioning

The strongest academic claim is not 'this is the first disaster resource allocation system.' That claim would be incorrect because emergency resource allocation has extensive prior research. Instead, state that the project develops and evaluates an implementable, explainable, web-based decision-support prototype that integrates multi-factor prioritization, supply constraints, distance-aware recommendations, human approval and comparative evaluation.

This wording makes the project defensible during a viva because it acknowledges prior research and clearly identifies what the student implementation contributes.

49. Conclusion

Disaster Resource Response Optimizer is a strong final-year Java full-stack project because it combines software engineering, database management, REST APIs, frontend development, security, algorithm design, optimization concepts and measurable research evaluation. The central feature should remain the allocation engine. The application should continuously connect disaster needs with available resources and provide an explainable recommendation rather than simply storing information.

The practical development strategy is to implement a reliable MVP first and then add research depth through baseline comparison, sensitivity analysis and evaluation. The system should be complete enough to demonstrate an end-to-end disaster scenario by 30 September 2026.

50. References and Research Basis

- Mukhopadhyay et al., A Review of Incident Prediction, Resource Allocation, and Dispatch Models for Emergency Management, Accident Analysis & Prevention, 2022.
- Wang, Multiperiod Optimal Allocation of Emergency Resources in Support of Cross-Regional Disaster Sustainable Rescue, International Journal of Disaster Risk Science, 2021.
- Multiperiod Equitable and Efficient Allocation Strategy of Emergency Resources Under Uncertainty, International Journal of Disaster Risk Science.
- Wu, Zou & Kang, Multi-stage robust and multi-role integrated allocation of emergency resources for hazard-induced disasters, Scientific Reports, 2026.
- Robust optimization of emergency resource location and coupling allocation considering multiple uncertainties, Socio-Economic Planning Sciences, 2026.
- Distributionally robust chance-constrained programming for multi-period emergency resource allocation and vehicle routing in disaster response operations, Omega.
- FEMA, Emergency Support Function #7 – Logistics Management and Resource Support Annex.
- FEMA, Response and Recovery / logistics and resource management materials.

51. Final Project Identity

Recommended one-line description for your project proposal:

“A web-based, explainable decision-support system that dynamically prioritizes disaster relief requests and recommends allocation of limited resources using severity, urgency, affected population, shortage and logistics constraints.”

Recommended short title: DRRO

Recommended stack: React + Spring Boot + MySQL

Recommended research focus: Explainable, dynamic, constraint-aware disaster resource allocation with baseline comparison.

Target completion: 30 September 2026

52. Source Notes for the Literature/Background Sections

The research and real-world background in this document was checked against current web sources in August 2026. The cited research indicates that emergency resource allocation is an active research area involving uncertainty, multi-period decisions, routing, equity and robust optimization. FEMA materials provide real-world context for logistics requirement/resource balancing and resource support.

- Scientific Reports (2026): Multi-stage robust and multi-role integrated allocation of emergency resources for hazard-induced disasters.
- Socio-Economic Planning Sciences (2026): Robust optimization of emergency resource location and coupling allocation considering multiple uncertainties.
- International Journal of Disaster Risk Science (2021): Multiperiod optimal allocation of emergency resources in cross-regional disaster rescue.
- Accident Analysis & Prevention (2022): Review of incident prediction, resource allocation and dispatch models for emergency management.
- FEMA: Emergency Support Function #7 – Logistics Management and Resource Support Annex.